

COURSE 3 - DATA ENGINEERING 2026**C3-073 - ShipmentStream Mini-Lab**

Integrated limited-hint streaming assessment on fresh parcel-event data.

Artifact	C3-073
Status	2026 - LEARNER EDITION
Phase	C3-06 Streaming & Event-Driven Pipelines 13 lessons 34-46 focused hours
Prerequisite	C3-05 Gate PASS; C2B streaming/lakehouse concepts may be recalled but Kafka/Spark streaming implementation is taught here.
Current lane	Apache Kafka 4.3.1 Spark Structured Streaming 4.2.0 Delta Lake 4.4.0 Debezium 3.6.2.Final
Brand	Charcoal #1F2937 Teal #14B8A6 Soft Blue #3B82F6 AMOLED #000

House teaching rule: mental picture -> tiny event example -> guided Kafka/streaming action -> expected result -> why it worked -> break/fix -> independent changed case -> validation -> replay/state/production reasoning -> evidence. A running stream is not mastery if duplicates, late data, offsets, checkpoints or sink state are wrong.

Scenario

ShipmentStream receives parcel status events. Build a replay-safe streaming design that preserves event identity, handles duplicates and late arrivals, produces event-time delivery windows, survives restart and can hand trustworthy state to a lakehouse sink.

Required build

- Inspect the supplied JSONL and state the event identity, entity key, event-time field and expected source count.
- Classify batch/micro-batch/event-driven requirements for the shipment alert and operations-report use cases.
- Design Kafka topic/partition key/group/retention/replay behavior; live Kafka proof is required only when the broker is actually run.
- Handle the duplicate event and explain the idempotent business-state boundary.
- Apply an explicit lateness/watermark policy to event-time windows and explain what happens to the supplied late event.
- Complete the Structured Streaming job scaffold; if PySpark is available, run it with a durable checkpoint and capture query progress.
- Prove same-checkpoint restart behavior and separately explain a fresh-checkpoint replay.
- Design Bronze/Silver sink contracts with separate checkpoints and a deterministic upsert/transaction boundary.
- Create a small operator dashboard/runbook covering lag/backlog, batch duration, state growth, checkpoint/sink failure and recovery.
- Save a README explaining the architecture, limits, exact runtime evidence obtained, and what remains simulated.

Pass standard

PASS only when every required competency is demonstrated and zero unresolved critical failures remain. Missing or fabricated runtime evidence is never repaired by stronger performance elsewhere.

Limited hints

- Use the simulator to reason about duplicate/late/checkpoint state, but label it as simulator evidence.
- Keep event time immutable; add ingest/processing timestamps separately.
- One checkpoint belongs to one streaming query identity.